

RoboBehaviors & Finite State Machine Project

ENGR 3590: Computational Intro to Robotics

Fall 2026 - Ellie Kung & Vanessa Flores

Introduction

In this project, we developed three different behaviors for a neato robot to execute within a finite state machine. Our team goals were to familiarize ourselves with ROS2, improve our python coding practices, and become more efficient at creating software architecture.

Behaviors

Collision Avoidance

The collision avoidance node functions as a safety partner that works in tandem with our other behavior scripts, rather than as a standalone behavior. It acts as an e-stop and prevents the neato from hitting obstacles that may be in its path. The node first evaluates LiDAR scans directly in front of it to detect whether an obstacle is near. As the neato approaches the obstacle, the velocities of the wheels are reduced and the neato is set still at a certain distance.

Additionally, the neato contains a built-in bump topic, of which our node is publishing to when one of the four bump sensors at the neato's front are hit. When a bump is detected, the neato will stop motion immediately.

Since the collision avoidance behavior does not create its own paths or wheel commands, it primarily serves as a mediator between the desired velocities and command velocities produced by other nodes. Every other behavior node publishes to a *des_vel* topic, which collision avoidance is subscribed to to ensure the neato is neither bumped or close to an obstacle. The node then publishes the command velocity, or *cmd_vel*, to the neato to execute motion.

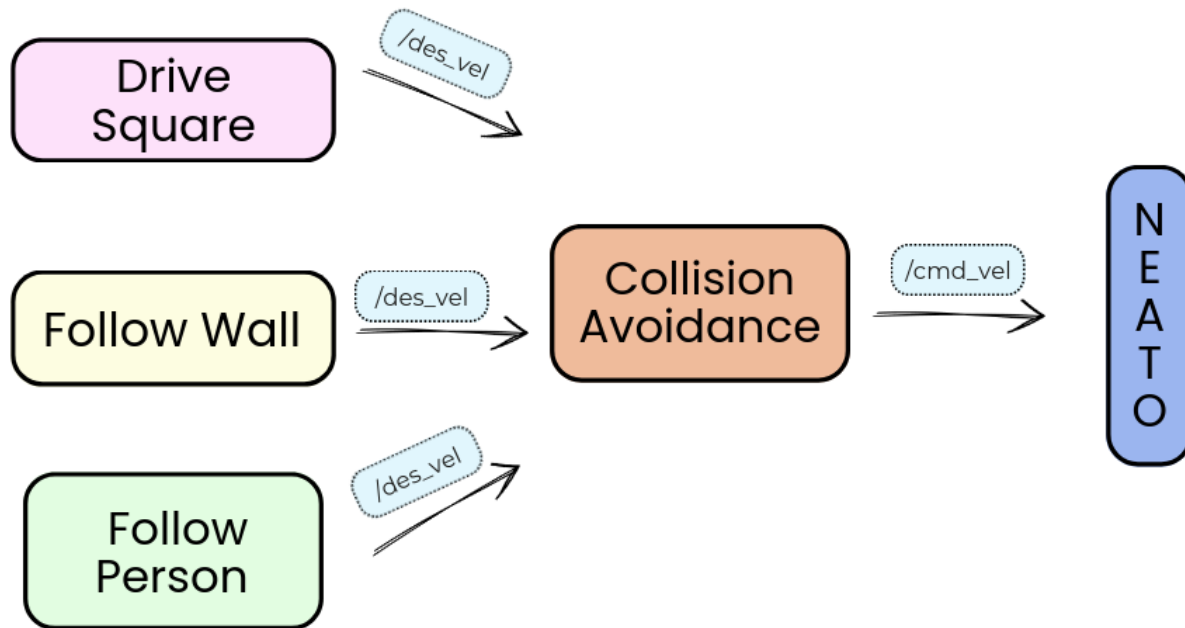


Figure 1: Diagram displaying the control flow between the three behaviors and collision avoidance. The node for drive square, wall following, and person following all publish desired velocities to the `des_vels` topic, which the collision avoidance node is subscribed to. This node then publishes the command velocities to the `neato`.

Drive Square

The goal of this behavior is to drive the neato in a predetermined shape. For simplicity, we chose to draw a square. We utilized the `Twist()` message type, which contains 2 vector components to represent our linear (forward/backward) and angular (left/right turn) velocities. Our performing loop consists of a for loop that commands the neato to drive forward for 0.8 meters and turn once, four times, making a square. The forward driving and turning functions are very simple, and simply state the speeds at which the neato is traveling.

Wall Following

The goal of this behavior is to enable a neato to find and follow a wall. To maintain an organized class structure, we implemented a mini state machine within the wall follower node, initially set to 'unknown'. While in this state, we process the LiDAR data to check if there are any walls within the threshold distance of 0.5m. If a wall is detected within this range, the internal state then transitions to 'at wall' state and checks if it is facing a wall by extracting two points and determining if the distance is the same. Otherwise, it will send angular velocity commands to rotate the neato and search for another wall.

Once the robot is at the wall, it will turn to align itself to be parallel with the wall by checking if the 80 degree and 100 degree points are equal. As seen in the figure below, once the distances are equal, the neato should be parallel to the wall.

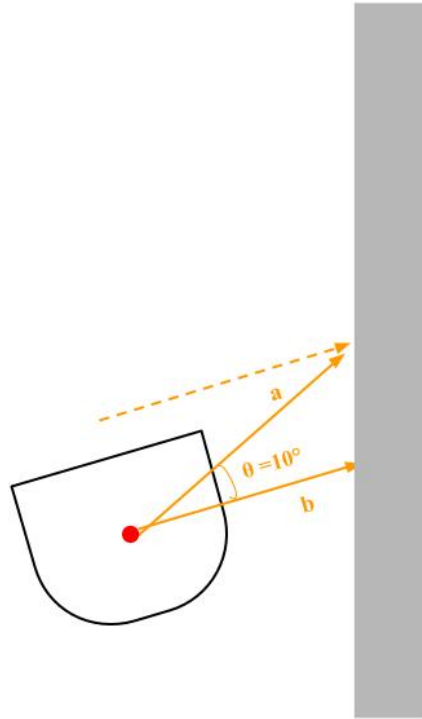


Figure 2: Diagram displaying geometric information used to create the algorithm that allows the robot to stay parallel to the wall during alignment and driving.

Once parallel to the wall, it implements a proportional wall follower. This is done by checking the LiDAR data directly to its left and the normalized distance of the angle 10 degrees forward from its left. If the first distance is larger, the robot turns away from the wall, otherwise the robot turns towards the wall. This is done to fix any differences between the neato's angle and the wall angle.

Additionally, the wall follower inspects the LiDAR scan in front of it to account when the neato reaches the corner of a room, where it can't see the wall using the left forward distance. In this case, the neato will start turning anyway, to prepare for the upcoming corner and larger turn.

Person Following

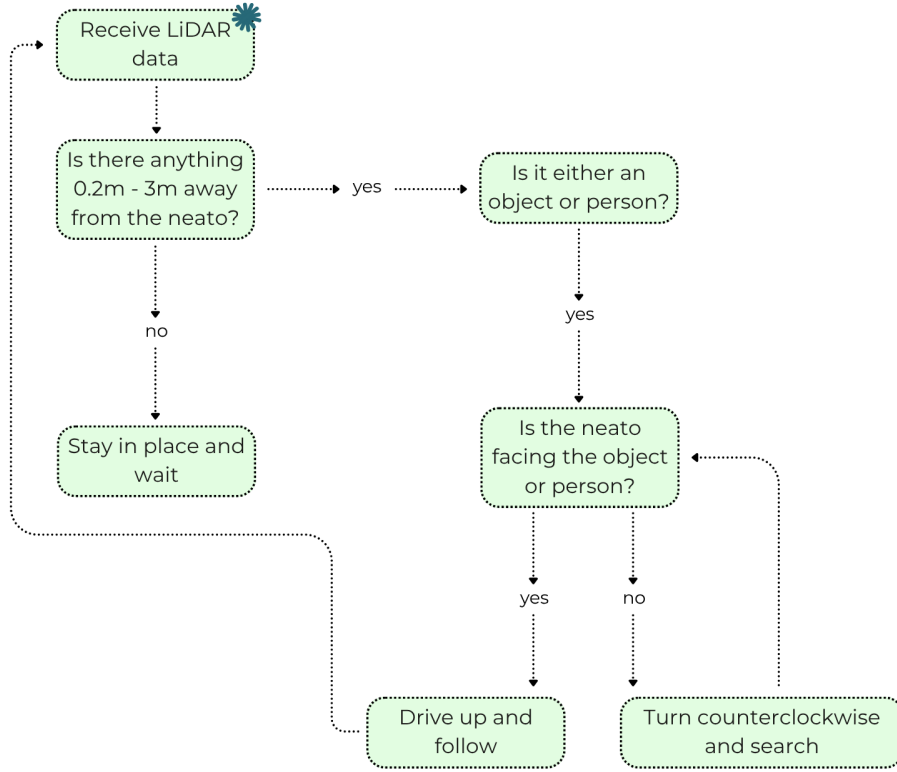


Figure 2: Diagram displaying the logical processing of the person following behavior. The cycle begins with receiving LiDAR data, which is then followed by data filtering, and lastly publishing velocities to the neato. The cycle is repeated once the velocities are published to ensure the neato is on track to its destination.

The goal of this behavior is to enable the neato to track and follow a moving object or person, as represented by a manually manipulated cube in our neato simulation. This node interfaces with the neato's LiDAR sensor by subscribing to the scan topic to receive incoming data, and publishing to the `cmd_vel` topic to send wheel velocities. Since we operated exclusively within a simulated environment and with a single object in the workspace, we assume that this behavior, deployed on a real neato, would require a cleared area free of obstacles with a single target. To manage target tracking within our node, we implemented four statuses, which we defined as "unknown", "found person", "looking at person", and "following person".

We begin to continuously processing and filter LiDAR data messages by iterating across the data using a conditional:

```
if self.person_dist2 < msg.ranges[i] < self.person_dist \
    and msg.ranges[i+5] < self.person_dist \
    and msg.ranges[i+10] < self.person_dist:
```

The conditional checks if the measurement at the index i falls within our predefined limits of 0.2m (*self.person_dist2*) and 3m (*self.person_dist*). To ensure the readings are describing a physical object or being rather than a single point or noise, the conditional also verifies that the adjacent entries $i + 5$ and $i + 10$ are below the 3m limit. If all three conditions are true, we can confirm that there is a target nearby, which changes the status from “unknown” to “found person”.

Once a target is recognized in the environment, we then shift to evaluating whether or not the neato is facing it.

```
if self.person_dist2 > msg.ranges[0] or self.person_dist < msg.ranges[0]:
```

This conditional checks the first entry of the data (*msg.ranges[0]*). If the reading is not within the set tracking limits, then the object is determined to be away from the neato’s front face. The system maintains the “found person” status and publishes an angular velocity command to rotate the neato in place until the neato is directly aligned with the target. Once the target is confirmed to be ahead, the status updates to “following person” and the node publishes a positive linear velocity command to drive forward towards the target.

Finite State Controller

Our neato, Bumpy the Ghost, is a finite state machine with three different states: driving in a square, following a wall, and following a person. He begins in the square driving state to examine and familiarize himself with his environment. Once he squares once, Bumpy finds a wall to follow in the hopes of finding someone to haunt. Once bumpy bumps into something (or someone?!) he will begin to haunt the object and follow them until he chases them down (and bumps into them again). This will reset Bumpy to the square driving state so that he can contemplate and think about his next steps.

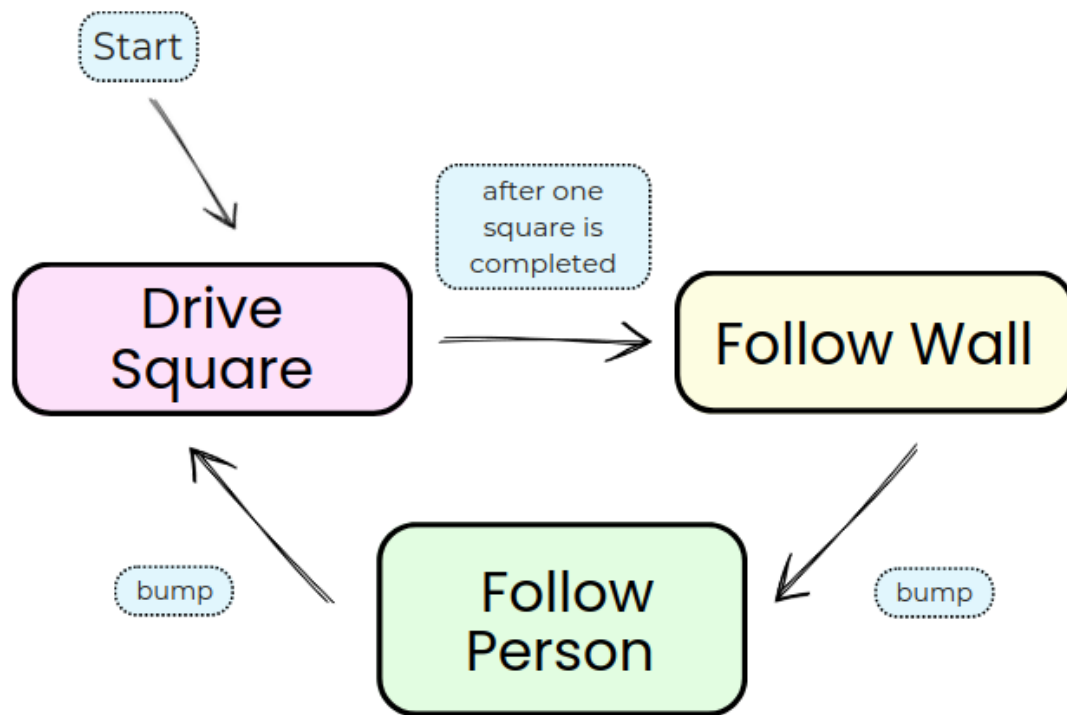


Figure 3: Diagram displaying the behavior switches of our system. The neato begins with driving in a square. After it completes one square, the state then switches to wall following. When the neato is bumped, it will then switch to the person following. When bumped again, the neato will restart the cycle, returning to square driving.

To coordinate state execution across our system, each behavior node contained its own variable that could be set to active (True) or inactive (False). A node will only publish desired velocities and perform its behavior if it is active. Each node is subscribed to a shared string /state topic, which publishes whenever a state is changed. When a node receives the state topic message, it checks to see if the string matches its identifier to determine whether or not it should activate or deactivate itself. For example, when the wall follower publishes ‘person_follower’ to the state topic, the wall follower node sets itself inactive while the person follower node becomes active and begins to send desired velocities.

Initially, our goal was to control the changing of states using input from the neato’s right and left bump sensors. However we encountered three main issues during the implementation. First, we were unable to manipulate the simulation objects accurately enough to bump only one side, therefore we allowed input from either sensor to be interpreted as the signal to switch states. Secondly, in the drive square state, the neato only processed bumps once the entire square was drawn because the code ran without threading. Lastly, because the bump topic publishes every tenth of a second while triggered, a single bump to us registered as multiple bumps to the neato, causing it to switch states rapidly. To combat this, we stopped the /bump subscription for the

person_follower for a few seconds so that any extra bumps during that time would not switch the state. However, this delay also means that there is a delay in the behavior starting. A better alternative would probably be to come up with an alternative state transition.

Architecture

To begin laying out our software architecture for the state machine, we first ideated which behaviors we wanted the neato to perform, then developed the general algorithms and published messages for each behavior.

Each distinct behavior was implemented as its own class, inheriting from the Node class. Within each behavior class, we configured publishers and subscribers to manage communication between nodes in the system. For instance, the collision_avoidance node subscribes to the */bump* and */scan* topics to monitor bumper taps and identify when the neato should stop. The same node publishes to the */cmd_vel* topic to communicate velocity commands to the neato.

Functionally, each class performs through a main loop or event-driven callback by subscriber data, which allows the nodes to run independently and indefinitely when needed.

Conclusion

Challenges

While sharing files, we encountered execution issues caused by Python metadata. Files created on one member's computer failed to run on another's, despite being created in the same package, leading us to create duplicates while troubleshooting. Once we traced the problem back to pycache, we deleted the stale files which instantly resolved the issue.

As mentioned before, we struggled to get the square drive to switch states immediately when bumped, instead the system processed a 'ghost' bump when the square was completed. We realized this was a threading issue and tried to implement it, but ultimately just changed the transition behavior all together.

We struggled to get the wall following to be fast and accurate. While the robot is capable of identifying and following a wall, it takes a while to find it and often makes really wide turns, especially when the corner turns out instead of turning in. This could likely be fixed if we took more LiDAR scans into account when processing. Even while it is following the wall, it often overcorrects and wobbles around. To fix this in the future, we would add a PID algorithm instead of what we have now.

Improvements

In the future we would develop more robust state transitions, as simulated bump sensor triggers can be inaccurate and cause the state to change more than once. Additionally, we would test our code on the physical neato robots to gain experience debugging with hardware and developing our scripts to be more accurate in a real world environment where conditions may not always be optimal.

While our wall finding process works, it is slow and slightly inefficient. With more time, we would have implemented a better algorithm to find the wall faster. Since the LiDAR scan includes all 360 degrees of the robot, we should have used the first scan to find the closest wall. At first, we weren't sure as to how to accurately get the robot to turn to the correct angle, because the local frame of the robot changes as it rotates and we couldn't identify where the robot was on the global frame. Similar idea for the person follower, we should be utilizing the entire LiDAR scan to figure out where the object is.

Learning Objectives

Ellie: My learning objective for this project was to get comfortable with ROS and getting reoriented to object oriented programming and python. I also wanted to get more comfortable with the simulator and visualization tools available to us as well as gaining intuition for how to write well structured behaviors and nodes for future use.

Vanessa: My learning goals for this project were to familiarize myself with ROS, develop a better understanding of how to create software architecture, and practice good python coding habits.

Key Takeaways

It is easy to overcomplicate nodes and functions when navigating ROS2 for the first time and coding complex behaviors. We found that the most effective approach to coding was to simplify the behavior as much as possible, breaking down each second of an operation into manageable steps. When something wasn't working and identifying the bug was difficult, we stepped back to look at our code and added print statements to detect where the code failed. This allowed us to study how the functions were interacting with each other and identify whether or not information was being correctly parsed and passed.